

Cheat sheet Business Analytics - Flaminia Neda - Netflix Project

#basic cleaning of data

```
# Look at the data
View(netflix_clean)
str(netflix_clean)
summary(netflix_clean)
# Drop a column that is not useful anymore
df$year <- NULL
# Keep only rows with a condition
df_actual <- subset(df, status == "actual")
df_actual <- df[df$status == "actual", ]
# Replace a spelling issue / typo
df$month <- sub("Mai", "May", df$month)
df[df$month == "Mai", "month"] <- "May"
```

#in case of duplicates

```
# Finding all the duplicates
duplicates <- duplicated(df)
sum(duplicates)
# in order to keep only duplicated rows for inspection
df_duplicates <- df[duplicates, ]
# Remove all full duplicates
df_no_duplicates <- df[!duplicated(df), ]
# Find partial duplicates
dup_partial <- duplicated(df[, c("product", "region", "month")])
df_partial_duplicates <- df[dup_partial, ]
# Show all duplicate entries incl. first occurrence
dup_all <- duplicated(df[, c("product", "region", "month")]) |
duplicated(df[, c("product", "region", "month")], fromLast = TRUE)
df_all_duplicates <- df[dup_all, ]
```

#if there are missing values (NA)

```
# Check where NA values are
is.na(df)
# Count the NAs per column
colSums(is.na(df))
# Keep rows that contain at least one NA
df_with_na <- df[rowSums(is.na(df)) > 0, ]
# Remove rows with NA
netflix_clean <- netflix_clean %>% filter(!is.na(release_year))
#if there are missing values
#replace missing values
netflix_clean$country[is.na(netflix_clean$country)] <- "Unknown"
# Calculate while ignoring NA
mean(df$value, na.rm = TRUE)
```

#reshaping of messy data

```
library(tidyr)
# to make in in a wider format: one variable becomes several columns
df_wide <- pivot_wider(df,
  names_from = type,
  values_from = value)
# For a longer format
df_long <- pivot_longer(df_wide,
  cols = Jan:Dec,
  names_to = "month",
  values_to = "value")
# Separate values in one cell into rows
df_clean <- df %>% separate_rows(country, sep = ",")
```

for quarto chunk settings

#everything done in the the right visual layout

```
```{r}
#| echo: false
#| warning: false
#| message: false
```

```
code here
...

```

## # Useful libraries for project

library(dplyr)

library(ggplot2)

library(tidyr)

library(readr)

library(gganimate)

## # data summaries from our project

```
library(dplyr)
library(ggplot2)
Movies vs TV Shows
netflix_clean %>%
count(type)
Top 5 producing countries, without Unknown
netflix_clean %>%
filter(!is.na(country), country != "Unknown", country != "") %>%
count(country, sort = TRUE) %>%
slice_head(n = 5)
Titles per year
titles_per_year <- netflix_clean %>%
filter(!is.na(release_year)) %>%
count(release_year)
```

## #Github

```
git add .
git commit -m "Project update"
git push
Purpose:
Version control and project sharing.
```

## # GGANIMATE

# Used to create animated visualizations that show changes over time.

# Required packages:

library(ggplot2)

library(gganimate)

# Example structure:

```
ggplot(data, aes(x, y)) +
 geom_col() +
 transition_states(time_variable)
```

# Example from Netflix project:

# Animated Top 5 Countries on Netflix

```
netflix_clean %>%
 filter(!is.na(country),
 country != "Unknown",
 country != "") %>%
 count(country, sort = TRUE) %>%
 slice_head(n = 5) %>%
 ggplot(aes(x = reorder(country, n), y = n)) +
 geom_col(fill = "#E50914") +
 coord_flip() +
 labs(
 title = "Top 5 Countries on Netflix",
 x = "Country",
 y = "Number of Titles"
) +
 theme_minimal()
```

# Common animation functions:

transition\_states() # Switches between states/categories

transition\_time() # Animates data over time

transition\_reveal() # Reveals lines or points progressively

.

## # Presentation design with css for quarto

# Used a custom CSS file (style.css) to create a Netflix-themed presentation.

# Added CSS file in YAML:

format:

revealjs:

theme: simple

css: style.css

slide-number: true

transition: fade

# Main CSS styling:

```
.reveal {
 background-color: #141414; # Netflix black
 color: white;
}
```

```
.reveal h1,
.reveal h2,
.reveal h3 {
 color: #E50914; # Netflix red
}
```

# CSS separates presentation design from analysis code,

# allowing easy customization of Quarto presentations.

## #Statistical tests + regression

# Normality test

shapiro.test(df\$variable)

# Chi-square test for two categorical variables

contingency\_table <- table(df\$gender, df\$program)

chisq.test(contingency\_table)

# Correlation test

cor.test(df\$x, df\$y, method = "pearson") # normal data

cor.test(df\$x, df\$y, method = "spearman") # non-normal / ranks

cor.test(df\$x, df\$y, method = "kendall") # smaller samples

# Linear regression

model <- lm(n ~ release\_year, data = titles\_per\_year)

summary(model)

# Visual regression line in ggplot

geom\_smooth(method = "lm", se = TRUE)